DRL Exam Cheat Sheet

Deep Reinforcement Learning (SS26)

1. Basics & MDPs

MDP: Tuple (S,A,P,R,γ). S: States, A: Actions, $P(s'|s,a)$: Trans. dynamics, $R(s,a)=\mathbb{E}[R|s,a]$: Reward, $\gamma\in[0,1]$.
Markov Property: $\mathbb{P}(S_{t+1},R_{t+1}|S_t,A_t,\dots) = \mathbb{P}(S_{t+1},R_{t+1}|S_t,A_t)$.
Return: $G_t=\sum_{k=0}^\infty \gamma^k R_{t+k+1}$.
Policy: $\pi(a|s)\in[0,1]$, $\sum_a \pi(a|s)=1$. Goal: $\max_\pi \mathbb{E}_\pi[G_t]$.
Value Funcs: $V^\pi(s)=\mathbb{E}_\pi[G_t|S_t=s]$, $Q^\pi(s,a)=\mathbb{E}_\pi[G_t|S_t=s,A_t=a]$.
Relation: $V^\pi(s)=\sum_a\pi(a|s)Q^\pi(s,a)$.
Optimal: $V^*(s)=\max_\pi V^\pi(s)$, $Q^*(s,a)=\max_\pi Q_\pi(s,a)$.
Advantage: $A^\pi(s,a)=Q^\pi(s,a)-V^\pi(s)$.
Bellman Expectation Equations:

$$V^\pi(s)=\sum_a \pi(a|s)\left[r(s,a)+\gamma\sum_{s'}P(s'|s,a)V^\pi(s')\right]$$

$$Q^\pi(s,a)=r(s,a)+\gamma\sum_{s'}P(s'|s,a)\sum_{a'}\pi(a'|s')Q^\pi(s',a')$$

Bellman Optimality Equations:

$$V^*(s)=\max_a\left[r(s,a)+\gamma\sum_{s'}P(s'|s,a)V^*(s')\right]$$

$$Q^*(s,a)=r(s,a)+\gamma\sum_{s'}P(s'|s,a)\max_{a'}Q^*(s',a')$$

2. Multi-Armed Bandits

$q(a)=\mathbb{E}[R_t|A_t=a]$. Sample avg: $Q_t(a)=\frac{1}{N_t(a)}\sum_{\tau\le t}R_\tau \mathbf{1}[A_\tau=a]$.
Incremental: $Q_{t+1}(a)=Q_t(a)+\frac{1}{N_t(a)}[R_t-Q_t(a)]$.
ϵ-greedy: $A_t=\arg\max_a Q_t(a)$ w.p. $1-\epsilon$.
UCB: $A_t=\arg\max_a\left[Q_t(a)+c\sqrt{\frac{\ln t}{N_t(a)}}\right]$.
Regret: $L_T=\sum_{t=1}^T[q^*-q(A_t)]$. ϵ-greedy gives linear regret. UCB yields logarithmic regret $O(\ln T)$.

3. Dynamic Programming

Requires known P,R. Based on sweeping Bellman equations. Prediction/policy evaluation (linear ⇒ closed form $O(|S|^3)$). Control/policy optimization (nonlinear ⇒ iterative).
Policy Eval: $V_{k+1}(s)=\sum_a\pi(a|s)\sum_{s',r}P[r+\gamma V_k(s')]$.
Policy Improv. Theorem: If $Q^\pi(s,\pi'(s))\ge V^\pi(s)\ \forall s$, then $V^{\pi'}(s)\ge V^\pi(s)\ \forall s$. Greedy: $\pi'(s)=\arg\max_a Q^\pi(s,a)$.
Policy Iteration: Evaluate V^π, then greedy improvement. Repeat. Guaranteed to converge.
Value Iteration: Combine eval/improve in one sweep: $V_{k+1}(s)=\max_a\sum_{s',r}P[r+\gamma V_k(s')]$. Extract optimal policy at the end.

4. Tabular Model-Free Prediction

MC: Full episodes. Unbiased, high variance. $V(S_t)\leftarrow V(S_t)+\alpha(G_t-V(S_t))$.
TD(0): 1-step bootstrap. Biased, lower variance. Online.

$$V(S_t)\leftarrow V(S_t)+\alpha\underbrace{(R_{t+1}+\gamma V(S_{t+1})-V(S_t))}_{\delta_t\text{ (TD error)}}$$

n-step TD: $G_t^{(n)}=\sum_{k=1}^n\gamma^{k-1}R_{t+k}+\gamma^nV(S_{t+n})$.
Update: $V(S_t)\leftarrow V(S_t)+\alpha(G_t^{(n)}-V(S_t))$.
Eligibility Traces TD(λ): Forward: $G_t^\lambda=(1-\lambda)\sum_{n=1}^\infty\lambda^{n-1}G_t^{(n)}$. (λ=0 ⇒TD(0), λ=1 ⇒MC.)
Backward: $z_t=\gamma\lambda z_{t-1}+\nabla V(S_t)$; $\Delta w=\alpha\delta_t z_t$.

5. Tabular Model-Free Control

SARSA (On-Policy TD): $Q(S_t,A_t)\leftarrow Q(S_t,A_t)+\alpha[R_{t+1}+\gamma Q(S_{t+1},A_{t+1})-Q(S_t,A_t)]$, ϵ-greedy for behavior + update. **GLIE:** $\epsilon_t\to0$, $\sum_t\epsilon_t=\infty$ ⇒ converges to q^*.
Expected SARSA: $Q\leftarrow Q+\alpha[R+\gamma\sum_a\pi(a|S')Q(S',a)-Q]$. Lower variance.
Q-Learning (Off-Policy TD): $Q(S_t,A_t)\leftarrow Q(S_t,A_t)+\alpha[R_{t+1}+\gamma\max_{a'}Q(S_{t+1},a')-Q(S_t,A_t)]$
Converges to q^*: Robbins-Monro ($\sum\alpha_t=\infty$, $\sum\alpha_t^2<\infty$)
Double Q-Learning: Decouples selection/evaluation to fix overestimation bias:
$Q^A\leftarrow Q^A+\alpha[R+\gamma Q^B(S',\arg\max_a Q^A(S',a))-Q^A]$.
Importance Sampling: $\rho_t=\pi(A_t|S_t)/\mu(A_t|S_t)$. Full traj: $\rho_t^T=\prod_{k=t}^{T-1}\rho_k$.

6. Value Function Approx & DQN

Scale to large state spaces. Approximate $Q_w(s,a)$ or $V_w(s)$.
Linear FA: $V_w(s)=w^Tx(s)$, update $\Delta w=\alpha\delta_t x(s)$. Unique global minimum.
Gradient Descent: MC: $\Delta w=\alpha(G_t-V_w(S_t))\nabla_wV_w(S_t)$. Linear MC → global opt.
Semi-Gradient Methods: Target depends on w, so not true gradient. TD(0): $\Delta w=\alpha\delta_t\nabla_wV_w(S_t)$. TD(λ): $\Delta w=\alpha(G_t^\lambda-V_w(S))\nabla_wV_w(S)$. Linear TD converges, non-linear can diverge.
DQN Objective: $L(w)=\mathbb{E}_\mathcal{D}[(Y-Q_w(s,a))^2]$
Target $Y=r+\gamma\max_{a'}Q_{w^-}(s',a')$.
DQN Core (two stabilization tricks):

1. *Experience Replay:* Random mini-batches from $\mathcal{D}$. Breaks temporal correlation, reuses data.
2. *Fixed Target Net (w^-):* $w^-\leftarrow\tau w+(1-\tau)w^-$. Stabilizes targets.

Rainbow (6 extensions on top of DQN):

3. *Double DQN:* $Y=r+\gamma Q_{w^-}(s',\arg\max_a Q_w(s',a))$. Fixes overestimation.
4. *Dueling Networks:* $Q(s,a)=V(s)+A(s,a)-\frac{1}{|A|}\sum_{a'}A(s,a')$.
5. *Prioritized Replay (PER):* $p_i\propto|\delta_i|^\omega$. Bias corrected via IS weights $w_i=(Np_i)^{-\beta}$.
6. *Distributional (C51):* Predict categorical distribution $Z(s,a)$ of returns.
7. *Multi-Step Returns:* n-step targets reduce bias, faster reward propagation.
8. *Noisy Nets:* $y=(\mu^w+\sigma^w\odot\epsilon^w)x+(\mu^b+\sigma^b\odot\epsilon^b)$. State-dep. exploration.

7. Policy Gradients I

Optimize $\pi_\theta(a|s)$ directly. Handles continuous/high-dim actions, enables stochastic policies.
Policy Objectives:

Episodic return (d_0: initial-state distribution):

$$J(\theta)=\mathbb{E}_{S_0\sim d_0,\pi_\theta}[G_0]=\mathbb{E}_{S_0\sim d_0}\left[\sum_a\pi_\theta(a|S_0)Q^{\pi_\theta}(S_0,a)\right]$$

Average reward (continuing tasks; d^{π_θ}: stationary dist.):

$$J(\theta)=\mathbb{E}_{\pi_\theta}[R_{t+1}]=\sum_s d_{\pi_\theta}(s)\sum_a\pi_\theta(a|s)\,r(s,a)$$

Log-Derivative Trick: $\nabla_\theta\pi_\theta(a|s)=\pi_\theta(a|s)\nabla_\theta\log\pi_\theta(a|s)$.
Policy Gradient Theorem (env. dynamics $p(s'|s,a)$ cancel out):

$$\nabla_\theta J(\theta)=\mathbb{E}_\pi\left[\nabla_\theta\log\pi_\theta(a|s)\,Q^\pi(s,a)\right]$$

Policy Gradient Learning Signals Ψ_t:
General parameter update:

$$\theta\leftarrow\theta+\alpha\gamma^t\Psi_t\nabla_\theta\log\pi_\theta(A_t|S_t)$$

where the learning signal Ψ_t can be substituted with:

G_t	REINFORCE (MC return, high variance)
$G_t-V_w(S_t)$	REINFORCE w/ baseline $b(s)$
$\delta_t=R_{t+1}+\gamma V_w(S_{t+1})-V_w(S_t)$	1-Step AC (TD(0))
$G_t^{(n)}-V_w(S_t)$	n-Step AC
$\hat A_t^{\text{GAE}(\lambda)}=\sum_{l=0}^\infty(\gamma\lambda)^l\delta_{t+l}$	TD(λ) / GAE (PPO)
$Q_w(S_t,A_t)$	Q-based AC (DDPG, SAC)

8. Actor-Critic & Stable Updates

Actor updates π_θ; Critic estimates V_w or Q_w (via $\Delta w=\alpha_w\Psi_t\nabla_wV_w$).
GAE Trade-offs: λ=0 ⟹ TD/high bias; λ=1 ⟹ MC/high variance.
KL Div.: $D_{\text{KL}}(\pi_{old}\|\pi_\theta)=\mathbb{E}_s\left[\sum_a\pi_{old}\log(\pi_{old}/\pi_\theta)\right]$.
NPG: $D_{\text{KL}}\approx\frac12\delta^\top F\delta$ (Fisher matrix F). $\Delta\theta=\alpha F^{-1}\nabla J$. Expensive (2nd order). Only local approx.
TRPO: $\max_\theta\mathbb{E}\left[\frac{\pi_\theta(a|s)}{\pi_{old}(a|s)}\hat A\right]$ s.t. $D_{\text{KL}}(\pi_{old}\|\pi_\theta)\le\epsilon$. Conj. grad., second order. Small, reliable policy updates.
PPO: $r_t(\theta)=\pi_\theta/\pi_{old}$. First-order, multiple epochs per batch. Limit destructive updates.

$$L^{\text{CLIP}}=\mathbb{E}\Big[\min\Big(r_t\hat A_t,\ \text{clip}(r_t,1-\epsilon,1+\epsilon)\hat A_t\Big)\Big]$$

Combined: $L^{\text{PPO}}=L^{\text{CLIP}}-c_1L^{\text{VF}}+c_2S[\pi_\theta]$ (entropy bonus).

9. Continuous Control

Gaussian Policy: $A_t\sim\mathcal{N}(\mu_\theta(S_t),\Sigma_\theta(S_t))$, score: $\nabla_\theta\log\pi_\theta=\frac{A_t-\mu_\theta}{\sigma^2}\nabla_\theta\mu_\theta$. See REINFORCE update.
DPG: Learn $Q_w(s,a)\approx Q^\mu(s,a)$. Deterministic policy $A_t=\mu_\theta(S_t)$. Chain rule through critic:

$$\nabla_\theta J(\theta)=\mathbb{E}\Big[\nabla_aQ_w(s,a)\big|_{a=\mu_\theta(s)}\nabla_\theta\mu_\theta(s)\Big]$$

DDPG (DQN+DPG): Off-policy, replay buffer, target nets w^-,θ^-. $a=\mu_\theta(s)$ (learned maximizer instead).
Critic: $\min_w\mathbb{E}[(r+\gamma Q_{w^-}(s',\mu_{\theta^-}(s'))-Q_w(s,a))^2]$.
Actor: $\max_\theta\mathbb{E}[Q_w(s,\mu_\theta(s))]$. Exploration via $\mathcal{O}$-U noise.
TD3 (fixes DDPG overestimation – 3 tricks):

1. *Clipped Double-Q:* $Y=r+\gamma\min_{i=1,2}Q_{\theta_i'}(s',\tilde a)$.
2. *Delayed Actor Updates:* Update μ_θ every d critic steps.
3. *Target Policy Smoothing:* $\tilde a=\mu_{\theta'}(s')+\text{clip}(\mathcal{N}(0,\tilde\sigma),-c,c)$.

SAC (Max-Entropy, off-policy, stochastic): $\mathcal{H}(\pi)=-\mathbb{E}[\log\pi]$.

$$J(\pi)=\mathbb{E}_\tau\left[\sum_t\gamma^t\left(R_{t+1}+\alpha\,\mathcal{H}(\pi(\cdot|S_t))\right)\right]$$

Soft Bellman: $Y=r+\gamma\mathbb{E}_{a'}[\min_iQ_{w_i^-}(s',a')-\alpha\log\pi(a'|s')]$.
Actor: $\max_\theta\mathbb{E}_s\left[\mathbb{E}_{a\sim\pi_\theta}[Q_w(s,a)]+\alpha\mathcal{H}(\pi_\theta(\cdot|s))\right]$.
Reparam.: $a=\mu_\theta(s)+\sigma_\theta(s)\odot\epsilon$, $\epsilon\sim\mathcal{N}(0,I)$ – gradients flow to θ.

10. Exploration & Representation

Bootstrap / Thompson Sampling: K Q-heads; sample one head per episode for acting. Bootstrap masks ensure data diversity across heads.
Directed Exploration: aug. reward $r_t=r_t^{\text{ext}}+\beta r_t^{\text{int}}$.
Count-Based: $r^{\text{int}}(s)\propto1/\sqrt{N(s)}$. Intractable in large spaces; use pseudo-counts or hash-based approximations.
ICM: Learn features ϕ via *inverse* model $\hat a_t=g(\phi(s_t),\phi(s_{t+1}))$ – filters dimensions the agent cannot control. Forward model then predicts next feature:

$$r_{\text{int}}=\|f(\phi(s_t),a_t)-\phi(s_{t+1})\|^2$$

Vulnerable to *noisy TV* (irreducible stochastic noise).
RND: Predictor f_θ is trained to match *fixed* random target f_{tgt}: Deterministic target ⇒ mitigates noisy TV.

$$r_{\text{int}}=\|f_\theta(s')-f_{\text{tgt}}(s')\|^2\quad(\downarrow\text{ as }s'\text{ is visited more})$$

Disagreement: $r_{\text{int}}\propto\text{Var}_k[\hat s_{t+1}^{(k)}]$ from dynamics model ensemble; high where model is uncertain.
State Rep. Learning ($o_t\to x_t$): Goals: Structure, Continuity, Sparsity. *Bisimulation:* Latent distance ∝ transition & reward similarity. *Aux. Tasks:* Add reconstruction or forward/inverse dynamics prediction. *Data Aug:* Crop/jitter to encourage invariance to visual nuisances. *Contrastive:* Pull similar states (+) close, push different (−) apart ($\mathcal{L}_{\text{NCE}}$). *Attention:* Mask/patch self-attention to filter visual distractors.

11. Credit Assignment

Three root causes: *Depth:* Long delay between action & reward → recency bias. *Density:* Sparse rewards → weak or absent learning signal. *Breadth:* Many contributing actions → diluted per-action credit.
Addressing Depth: n-step returns / TD(λ) bridge short and long horizons. GAE: exponential weighting of δ_t's, $\hat A_t^{\text{GAE}}=\sum_l(\gamma\lambda)^l\delta_{t+l}$. RUDDER: learn return predictor $\hat G_t$; redistribute $\tilde r_{t+1}=\hat G_{t+1}-\hat G_t$.
Addressing Density: Reward shaping.
Addressing Breadth: Advantage $A(s,a)=Q-V$ isolates the contribution of action a over the average.
HER: Goal-conditioned policy; relabel failed traj. $(s_0,\dots,s_T)$ with achieved goal $g'=s_T$. Every episode becomes a success for *some* goal ⇒ dense signal in sparse-reward envs. Lim: requires a goal-conditioned MDP.

12. Imitation Learning (IL)

BC (Behavioral Cloning): Supervised MLE on expert demos:

$$\max_\theta\mathbb{E}_{(s,a)\sim\mathcal{D}_{\text{exp}}}[\log\pi_\theta(a|s)]$$

Flaw – Distribution Shift: agent deviates → visits states absent from $\mathcal{D}$ → errors compound. *Ross & Bagnell 2010:* $J(\pi^*) - J(\hat\pi_{\mathrm{BC}}) \le T^2\epsilon$ where $\epsilon = \max_s \mathcal{L}^{\pi^*}(s)$. Quadratic in horizon T.

DAgger (Dataset Aggregation): Trains on learner's *own* distribution. Reduces bound to $O(T\epsilon)$.

1. Init $\mathcal{D} \leftarrow \mathcal{D}_{\exp}$. Train $\hat\pi_1$ via BC.
2. **Repeat:** Roll out $\hat\pi_i$, collect visited states $\mathcal{S}_i$.
3. Query expert: get labels $\pi^*(s)\ \forall s \in \mathcal{S}_i$.
4. $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathcal{S}_i, \pi^*(\mathcal{S}_i))\}$; retrain $\hat\pi_{i+1}$.

Other Failure Modes: *Non-Markovian expert:* Expert uses history ⇒ Markov BC policy is confused. Fix: use RNN / memory. *Causal confusion:* Policy latches on spurious correlates (e.g. brake light, not velocity). *Multimodal actions:* MSE regressor averages modes → unsafe mean action. Fix: diffusion/flow policies or mixture density networks.

13. Offline RL

Fixed dataset $\mathcal{D} = \{(s, a, r, s')\}$ from behaviour policy π_β; no env interaction.

Core Challenge: Bootstrapping $Q(s', a')$ for OOD actions $a' \notin \mathcal{D}$ causes overestimation that propagates via Bellman ⇒ policy collapse.

Behavior Reg: Constrain $\pi \to \pi_\beta$ to avoid OOD errors: $\max J(\pi)$ s.t. $D(\pi\|\pi_\beta) \le \delta$. *Methods:* **1)** KL Penalty (λD_{KL}), **2)** BC Aux Loss ($\lambda\mathcal{L}_{\mathrm{BC}}$), **3)** Support Bound ($\pi(a|s) = 0$ if OOD).

CQL (Conservative Q-Learning): Explicitly penalise OOD Q-values while supporting in-data ones:

$$\min_Q\ \underbrace{\alpha(\mathbb{E}_{s,a\sim\mu}[Q] - \mathbb{E}_{s,a\sim\mathcal{D}}[Q])}_{\text{conservative penalty}} + \underbrace{\tfrac{1}{2}\mathcal{L}_{\mathrm{TD}}}_{\text{Bellman error}}$$

Provably conservative: $Q_{\mathrm{CQL}}(s,a) \le Q^\pi(s,a)\ \forall(s,a)$.

IQL: Avoids OOD queries entirely. Replaces $\max_a$ with expectile regression: $V(s) = \arg\min_V \mathbb{E}_{a\sim\mathcal{D}}[\mathcal{L}^\tau(Q(s,a) - V(s))]$. *Loss:* $\mathcal{L}^\tau(u) = |\tau - \mathbf{1}[u<0]|u^2$. For $\tau \approx 1$, $V(s) \approx \max_{a\in\mathcal{D}} Q(s,a)$ (soft max over in-data actions).

Decision Transformer (DT): Return-to-go $\hat R_t = \sum_{t'\ge t} r_{t'}$. GPT autoregressively models $(\hat R_t, s_t, a_t)$ sequences; at test time condition on desired $\hat R_0$. *No Bellman*: trained purely supervised. *No stitching*: cannot combine sub-trajectories from $\mathcal{D}$ to find better paths (unlike CQL/IQL).

14. Inverse RL & RLHF

IRL: Recover $R_\psi(s,a)$ from expert demos $\mathcal{D}_{\exp}$. Ill-posed (R=0 trivially optimal).

Max-Margin IRL: Expert must beat all policies by margin $D(\pi_{\exp}, \pi)$:

$$\min_\psi \|\psi\|^2 \text{ s.t. } \psi^\top\mu(\pi_{\exp}) \ge \psi^\top\mu(\pi) + D(\pi_{\exp}, \pi)\ \forall\pi$$

Max-Entopy IRL: $P(\tau) \propto \exp(r_\psi(\tau))$. MLE objective – match feature expectations:

$$\nabla_\psi L = \underbrace{\mathbb{E}_{\mathcal{D}_{\exp}}[\nabla_\psi r_\psi]}_{\text{expert features}} - \underbrace{\mathbb{E}_{P(\tau)}[\nabla_\psi r_\psi]}_{\text{model features}}$$

Requires solving RL in inner loop; handles stochastic/suboptimal experts.

GAIL: Adversarial IRL. Jointly optimise:

$$\min_{\pi_\theta}\max_{D_\psi}\ \mathbb{E}_{\pi_{\exp}}[\log D_\psi(s,a)] + \mathbb{E}_{\pi_\theta}[\log(1 - D_\psi(s,a))]$$

Reward $r(s,a) = -\log(1 - D_\psi(s,a))$; update π_θ via PPO/TRPO.

RLHF:

1. *SFT:* Fine-tune pretrained LM on curated demos.
2. *Collect preferences:* Human feedback.
3. *Reward Model:* $P(y_w \succ y_l) = \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$. Train via BCE: $\max_\phi \mathbb{E}[\log P(y_w \succ y_l)]$.
4. *RL fine-tune* (PPO) with KL regularisation to prevent reward hacking: $\max_\pi \mathbb{E}_{(x,y)\sim\pi}[r_\phi(x,y)] - \beta D_{\mathrm{KL}}(\pi\|\pi_{\mathrm{SFT}})$.

RLAIF: Feedback from AI instead of humans.

DPO: Skips explicit reward model. Reparametrise optimal reward via policy ratio: $r^*(x,y) = \beta\log\frac{\pi^*(y|x)}{\pi_{\mathrm{SFT}}(y|x)} + \beta\log Z(x)$. Substituting into BT loss and dropping $Z(x)$ (cancels):

$$L_{\mathrm{DPO}} = -\mathbb{E}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{\mathrm{SFT}}(y_w|x)} - \beta\log\frac{\pi_\theta(y_l|x)}{\pi_{\mathrm{SFT}}(y_l|x)}\right)\right]$$

One-stage supervised; no RL loop.

Goodhart's Law: proxy reward ≠ true reward at optimum. Mitigate: KL penalty, ensemble r_ϕ, diverse evals.

15. Planning & Search (Known Model)

MPC (Receding Horizon): Solve $\max_{a_{0:H}} \sum_{t=0}^{H-1}\gamma^t r(s_t, a_t)$; execute a_0^* only; re-plan from new s_1. Corrects model errors online.

CEM (Cross-Entropy Method): Gradient-free continuous optimiser. Init μ_0=0, Σ_0=$\sigma^2 I$. Each iter: (1) sample N seqs $a_{0:H}^{(i)} \sim \mathcal{N}(\mu_k, \Sigma_k)$; (2) simulate, score; (3) keep top-E elites; (4) refit μ_{k+1}, Σ_{k+1}. Can warm-start from previous μ.

MCTS: 4 phases; builds asymmetric tree toward promising regions.

1. *Selection* (UCT): $a^* = \arg\max_a\left[Q(s,a) + c\sqrt{\frac{\ln N(s)}{N(s,a)}}\right]$.
2. *Expansion:* Add new child node at leaf.
3. *Simulation:* Rollout with fast policy to terminal.
4. *Backpropagate:* $N(s,a)$+=1; $Q(s,a) \leftarrow Q + \frac{1}{N}(G - Q)$.

Play: $a^* = \arg\max_a N(s_0, a)$ (visit count, not Q).

AlphaGo: MCTS guided by SL+RL policy net p_σ, fast rollout p_π, value net v_θ. *Modified UCT* (PUCT): $a^* = \arg\max_a\left[Q(s,a) + c\frac{p_\sigma(a|s)}{1+N(s,a)}\right]$.

Leaf evaluation: $V(s_L) = (1-\lambda)v_\theta(s_L) + \lambda z$, where $z \in \{-1, 0, +1\}$ is rollout outcome.

AlphaZero: Single net $f_\theta(s) = (p, v)$; self-play only; no SL, no rollout policy.

MCTS produces improved policy: $\pi_{\mathrm{MCTS}}(a|s) \propto N(s,a)^{1/\tau}$ ($\tau \to 0$: greedy; τ=1: proportional). Training targets $(s_t, \pi_{\mathrm{MCTS}}(\cdot|s_t), z)$; loss:

$$L = \underbrace{(v_\theta(s) - z)^2}_{\text{value}} - \underbrace{\pi_{\mathrm{MCTS}}^\top\log p_\theta}_{\text{policy (cross-ent.)}} + \underbrace{c\|\theta\|^2}_{\text{regularization}}$$

16. Model-Based RL (Learned Model)

Learn $\hat P_\phi(s_{t+1}|s_t, a_t)$, $\hat R_\phi(s,a)$ from interactions.

Trade-off: Better sample efficiency; but distributional shift + compounding error ($\le H\cdot\epsilon_\phi$ per step) + model exploitation can hurt asymptotic performance.

Dyna-Q: Interleave real and imagined updates each timestep.

1. Act, observe (s, a, r, s'); store in $\mathcal{D}$; update model $\hat P_\phi$.
2. Real update: $Q \leftarrow Q + \alpha[r + \gamma\max_{a'} Q(s',a') - Q(s,a)]$.
3. Repeat $K\times$: sample $(s,a) \sim \mathcal{D}$, $(r', \hat s') \sim \hat P_\phi$, update Q.

K=0: Q-learning. Large K: fast convergence if model accurate.

Dyna-Q+: Adds an exploration bonus. $r^+ = r + \kappa\sqrt{\tau}$

MBPO: Ensemble $\hat P_\phi$; branch short k-step rollouts (k=1–5) from real $s_0 \sim \mathcal{D}_{\mathrm{real}}$; train SAC on mixed buffer $\mathcal{D}_{\mathrm{real}} \cup \mathcal{D}_{\mathrm{model}}$. Limits compounding error; ∼20× sample efficiency.

Probabilistic Ensembles (PETS): M Gaussian models $\hat P_{\phi_i}(s'|s,a) = \mathcal{N}(\mu_{\phi_i}, \Sigma_{\phi_i})$.

$$\mathrm{Var}_{\mathrm{ens}} = \underbrace{\frac{1}{M}\sum_i \Sigma_{\phi_i}}_{\text{aleatoric (env. noise)}} + \underbrace{\frac{1}{M}\sum_i(\mu_{\phi_i} - \bar\mu)^2}_{\text{epistemic (model unc.)}}$$

Use epistemic uncertainty for exploration or pessimistic planning.

Latent World Models: Encode high-dim obs. to compact z_t.

Dreamer: AC (actor π_θ, critic V_ψ) trained entirely in imagination by backpropagating through RSSM dynamics; no real env steps during policy update.

MuZero: No observation decoder. Learns *value-equivalent* model: representation $h_t = e(o_t)$; dynamics $(r, h_{t+1}) = g(h_t, a_t)$; prediction $(p, v) = f(h_t)$. MCTS in latent h space. Only requires accurate value/policy/reward, not s' reconstruction.

17. RL for Foundation Models

LLM as Policy: $\pi_\theta(a_t|s_t) = p_\theta(y_t|x, y_{<t})$. State = prompt + prefix; action = next token/tool call; reward = preference/verifier/task success. Challenges: huge $|\mathcal{A}|$ (∼10^5 tokens), long horizons, sparse seq-level rewards, conflicting objectives, distribution shift.

Pretraining (BC from text): $\max_\theta \sum_{(x,y)\in\mathcal{D}}\sum_t \log\pi_\theta(y_t|x, y_{<t})$. Imitates data distribution; not the same as being a helpful assistant.

SFT: Same MLE objective on curated demos $(x, y^*) \sim \mathcal{D}_{\mathrm{demo}}$. Simple & stable; copies behaviour rather than maximising preference.

RLHF: (1) SFT. (2) Bradley-Terry reward model from human pref. pairs $y_w \succ y_l$: $\mathcal{L}_{\mathrm{RM}} = -\mathbb{E}[\log\sigma(r_\phi(x, y_w) - r_\phi(x, y_l))]$. (3) KL-regularised RL:

$$\max_\theta \mathbb{E}_{y\sim\pi_\theta}\left[r_\phi(x,y) - \beta\log\frac{\pi_\theta(y|x)}{\pi_{\mathrm{ref}}(y|x)}\right]$$

KL penalty (π_θ vs π_{ref}) = stay close to SFT globally; PPO clipping (π_θ vs $\pi_{\theta_{\mathrm{old}}}$) = stable local update.

GRPO: No value function. Sample G outputs per prompt, score with r_ϕ, normalise within group: $\hat A_i = \frac{r_i - \mathrm{mean}(r_{1:G})}{\mathrm{std}(r_{1:G}) + \delta}$. Apply $\hat A_i$ to all tokens of y_i as in PPO-clip. Saves memory vs PPO+critic.

RLHF Failure Modes: Reward hacking (proxy ≠ intent); overoptimisation (quality drops after too many steps); preference misspec. (style ≻ correctness); mode collapse (generic safe answers); RM distribution shift. Mitigate: KL penalty, ensemble r_ϕ, early stopping.

RLVR: Replace learned r_ϕ with automatic checker (math verifier, code tests, game score). Composite reward: $r(y) = \lambda_{\mathrm{fmt}} r_{\mathrm{fmt}}(y) + \lambda_{\mathrm{cor}} r_{\mathrm{cor}}(y)$.

	Signal	Strength	Risk
SFT	demos	stable/cheap	limited by examples
RLHF	human prefs	subj. quality	proxy hacking
DPO	pref. pairs	simple/stable	offline limits
RLVR	verifier	scalable	narrow/verifier hacking

Generalist Agents: Tokenise text, images, state, actions into shared sequence ⇒ LM and action prediction structurally identical. **Gato:** single Transformer for 600+ tasks via supervised seq. modelling on mixed offline data.

18. Taxonomies & Comparisons

Deadly Triad: FA + Bootstrapping + Off-Policy ⟹ can diverge.

Aleatoric uncertainty: inherent randomness in env.

Epistemic uncertainty: due to limited knowledge.

	DP	MC	TD
Bootstraps?	Yes	No	Yes
Samples?	No	Yes	Yes
Model req.?	Yes	No	No

	NPG	TRPO	PPO
Policy type	On-policy	On-policy	On-policy
Constraint	Implicit KL	Hard KL	Soft KL
Key idea	Natural grad.	Trust region	Clipping

	PPO	TD3	SAC
Policy	Stochastic	Deterministic	Stochastic
Data	On-policy	Off-policy	Off-policy
Explore	Entropy	Added noise	Entropy obj.
Best Fit	Stable, large-scale	Cont. control baseline	Sample-efficient
Use Cases	Sims, LLMs, games	Benchmarks, precision	Robotics, real-world
Trade-off	Data hungry	Weak exploration	More moving parts

Method	Family	OOD	Stitching
BC Reg	Behavior reg.	Avoided	Limited
CQL	Conservative Q	Penalized	Yes
IQL	Conservative	Avoided	Yes
DT	Seq. modeling	Avoided	No

	MCTS	CEM/RS
Actions	Discrete	Continuous
Object	Tree	Sequence
Guidance	UCT+nets	Elite selection

On-policy	REINFORCE, SARSA, PPO
Off-policy	Q-Learning, DQN, DDPG, TD3, SAC
Offline	CQL, IQL, DT, BC